Name: Miheeka Kewalramani
Roll No.45 Branch:AI&DS Class:T12 Subject:ASD&D

Experiment 01

Aim: Version Control Operations using GIT

This experiment maps to LO:

LO2: Use Git effectively for version control, source management, and collaborative team development

Theory:

Version Control System (VCS) is a software tool used to manage and track changes made to files over time. It allows developers to maintain different versions of a project, restore previous versions, and collaborate efficiently with other team members.

Git is a distributed version control system widely used for source code management. It stores the complete history of changes locally and allows developers to work independently using branches. Git helps in tracking modifications, maintaining project integrity, and managing multiple versions of software projects.

A Git repository contains all project files along with their change history. The basic workflow in Git involves:

- **Working Directory:** The location where files are created and modified.
- **Staging Area:** A temporary area where changes are prepared before committing.
- **Repository:** The database where committed changes and project history are stored.

Git operations such as git add, git commit, and git log help in tracking and managing changes. Branching allows developers to create separate development paths without affecting the main project, while merging combines changes from different branches.

Git can also be connected to remote platforms such as GitHub, which provides cloud-based repository hosting and enables collaboration among developers. Commands like git push, git pull, and git fetch are used for synchronization between local and remote repositories.

Rollback operations such as git restore, git reset, and git revert help recover previous versions of files and undo unwanted changes.

Thus, Git provides an efficient environment for version control, source management, and collaborative software development.

Procedure:

1. **Checked the installed Git version.**

   git --version

```
Lab202@LAB05-021 MINGW64 ~
$ git --version
git version 2.55.0.windows.2
```

2. **Configured the Git username.**

   git config --global user.name "Your Name"

```
Lab202@LAB05-021 MINGW64 ~
$ git config --global user.name "miheeka27"
```

3. **Configured the Git email.**

   git config --global user.email "your_email@example.com"

```
Lab202@LAB05-021 MINGW64 ~
$ git config --global user.name "miheeka2006@gmail.com"
```

4. **Created a new project directory.**

   mkdir GitLab

```
Lab202@LAB05-021 MINGW64 ~
$ mkdir GitLab
```

5. **Navigated to the project directory.**

   cd GitLab

```
Lab202@LAB05-021 MINGW64 ~
$ cd GitLab
```

6. **Initialized a new Git repository.**

   git init

```
Lab202@LAB05-021 MINGW64 ~/GitLab
$ git init
Initialized empty Git repository in C:/Users/Lab202/GitLab/.git/
```

7. **Created a new text file.**

   echo "Hello Git" > file1.txt

```
Lab202@LAB05-021 MINGW64 ~/GitLab (master)
$ echo "Hello Git">file1.txt
```

```
Lab202@LAB05-021 MINGW64 ~/GitLab (master)
$ cat file1.txt
Hello Git
```

8. **Checked the repository status.**

   git status

```
Lab202@LAB05-021 MINGW64 ~/GitLab (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        file1.txt

nothing added to commit but untracked files present (use "git add" to track)
```

9. **Added the file to the staging area.**

   git add file1.txt

```
Lab202@LAB05-021 MINGW64 ~/GitLab (master)
$ git add file1.txt
warning: in the working copy of 'file1.txt', LF will be replaced by CRLF the next time Git touches it
```

10. **Verified the staging status.**

    git status

```
Lab202@LAB05-021 MINGW64 ~/GitLab (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   file1.txt
```

11. **Committed the staged changes.**

    git commit -m "First commit"

```
Lab202@LAB05-021 MINGW64 ~/GitLab (master)
$ git commit -m "First commit"
[master (root-commit) 046bba5] First commit
 1 file changed, 1 insertion(+)
 create mode 100644 file1.txt
```

12. **Displayed the commit history.**

    git log

```
Lab202@LAB05-021 MINGW64 ~/GitLab (master)
$ git log
commit 046bba5afad84c7f04588061b75f30aba3a9a6b5 (HEAD -> master)
Author: miheeka2006@gmail.com <>
Date:   Tue Jul 14 10:19:26 2026 +0530

    First commit
```

13. **Created a new branch named feature.**

    git branch feature

```
Lab202@LAB05-021 MINGW64 ~/GitLab (master)
$ git branch feature
```

14. **Listed all available branches.**

    git branch

```
Lab202@LAB05-021 MINGW64 ~/GitLab (master)
$ git branch
  feature
* master
```

15. **Switched to the feature branch.**

    git checkout feature

```
Lab202@LAB05-021 MINGW64 ~/GitLab (master)
$ git checkout feature
Switched to branch 'feature'
```

16. **Modified the existing file.**

    echo "New line" >> file1.txt

```
Lab202@LAB05-021 MINGW64 ~/GitLab (feature)
$ echo "New line">>file1.txt
```

17. **Checked the repository status again.**

    git status

```
Lab202@LAB05-021 MINGW64 ~/GitLab (feature)
$ git status
On branch feature
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   file1.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

18. **Added the modified file to the staging area.**

    git add file1.txt

```
Lab202@LAB05-021 MINGW64 ~/GitLab (feature)
$ git add .
warning: in the working copy of 'file1.txt', LF will be replaced by CRLF the next time Git touches it
```

19. **Committed the updated file.**

    git commit -m "Updated file"

```
Lab202@LAB05-021 MINGW64 ~/GitLab (feature)
$ git commit -m "Updated file"
[feature e840d47] Updated file
 1 file changed, 1 insertion(+)
```

20. **Switched back to the main branch.**

    git checkout master

```
Lab202@LAB05-021 MINGW64 ~/GitLab (feature)
$ git checkout master
Switched to branch 'master'
```

21. **Merged the feature branch into the main branch.**

    git merge feature

```
Lab202@LAB05-021 MINGW64 ~/GitLab (master)
$ git merge feature
Updating 046bba5..e840d47
Fast-forward
 file1.txt | 1 +
 1 file changed, 1 insertion(+)
```

22. Displayed the commit history in one-line format.

git log --oneline

```
Lab202@LAB05-021 MINGW64 ~/GitLab (master)
$ git log --one line
fatal: ambiguous argument 'line': unknown revision or path not in the working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'
```

23. Deleted the merged feature branch.

git branch -d feature

```
Lab202@LAB05-021 MINGW64 ~/GitLab (master)
$ git branch -d feature
Deleted branch feature (was e840d47).
```

26. Connected the local repository with GitHub remote repository.

git remote add origin <repository-url>

```
Lab202@LAB05-021 MINGW64 ~/GitLab (master)
$ git remote add origin https://github.com/miheeka27/lab
```

27. Checked the remote repository connection.

git remote -v

```
Lab202@LAB05-021 MINGW64 ~/GitLab (master)
$ git remote -v
origin  https://github.com/miheeka27/lab (fetch)
origin  https://github.com/miheeka27/lab (push)
```

28. Renamed the branch to main.

git branch -M main

```
Lab202@LAB05-021 MINGW64 ~/GitLab (master)
$ git branch
* master

Lab202@LAB05-021 MINGW64 ~/GitLab (master)
$ git branch -M main
```

29. Uploaded the local repository to GitHub.

git push -u origin main

```
Lab202@LAB05-021 MINGW64 ~/GitLab (main)
$ git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (6/6), 438 bytes | 438.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/miheeka27/lab
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

30. Pushed local commits to GitHub repository.

git push origin main

```
Lab202@LAB05-021 MINGW64 ~/GitLab (main)
$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 310 bytes | 310.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/miheeka27/lab
   c0dc3e2..6dd286a  main -> main
```

31. Downloaded changes from GitHub without merging.

git fetch origin

```
Lab202@LAB05-021 MINGW64 ~/GitLab (main)
$ git fetch origin
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 2 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (2/2), 903 bytes | 90.00 KiB/s, done.
From https://github.com/miheeka27/lab
   8c7f141..c0dc3e2  main       -> origin/main
```

32. Downloaded and merged changes from GitHub.

git pull origin main

```
Lab202@LAB05-021 MINGW64 ~/GitLab (main)
$ git pull origin main
From https://github.com/miheeka27/lab
 * branch            main       -> FETCH_HEAD
Updating e840d47..c0dc3e2
Fast-forward
 imp     | 1 +
 imp.txt | 1 +
 2 files changed, 2 insertions(+)
 create mode 100644 imp
 create mode 100644 imp.txt
```

33. Cloned a repository from GitHub.

git clone <repository-url>

```
Lab202@LAB05-021 MINGW64 ~/GitLab (main)
$ git clone https://github.com/codedex-io/github-chapter-2-sandbox.git
Cloning into 'github-chapter-2-sandbox'...
remote: Enumerating objects: 3, done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 3 (from 1)
Receiving objects: 100% (3/3), done.
```

34. Displayed local and remote branches.

git branch -a

```
Lab202@LAB05-021 MINGW64 ~/GitLab (main)
$ git branch -a
* main
```

35. Discarded uncommitted changes in a file.

git restore file1.txt

```
Lab202@LAB05-021 MINGW64 ~/GitLab (main)
$ git restore file1.txt
```

Conclusion:

I learned how to use Git commands for version control, including initializing repositories, adding and committing files, viewing history, creating branches, merging changes, and managing remote repositories using GitHub. I also learned commands like `git push`, `git pull`, `git fetch`, and `git clone` for synchronizing code with remote repositories, along with rollback operations to manage changes. This experiment helped me understand how Git is used for efficient source management and collaborative development.